

ジャッジニア

AIが当たり前の時代に、
根拠を持って判断できるエンジニアになる
経験を積む場所

背景

AIに、丸投げしていませんか？

調べるのも、直すのも、AIに聞けばすぐ答えが返ってきます。

44.1%

AIでエラー原因を調べる時間が減った

スーパーソフトウエア調査, 2026年8月

背景

問題が起きたとき、決めるのは自分

運用中のサービスで問題が起きたら、直し方を決めて責任を持つのは自分です。
「AIがそう言ったから」は通用しません。

91.5%

最終的に人が責任を持つべき仕事がある

スーパーソフトウエア調査, 2026年8月

背景

直し方に、正解はひとつじゃない

顧客データを取引先に渡している会社で、データが漏れるおそれのある欠陥が見つかったら？

案1

すぐ直せる
取引先へのデータ提供が止まる

案2

取引先に影響しない
直すのに時間がかかる

取引の重さまでは、AIは知りません。決めるのは人です。

背景

判断の力は、経験でしか身につかない

でも、その経験を積める場は限られています。

39.9%

判断できる人が
3人以下の職場

68.0%

その人がいないと
対応が難しい

障害対応をするエンジニア316人 ／ PagerDuty調査, 2026年8月

課題

課題をまとめると

エンジニア

AIで作れても、
判断した経験がない

企業

新しく入った人に、
経験を積ませる余裕がない

だから、本番の前に判断を
自分で経験できる場所を作りました。

ジャッジニアとは

公開後に起きることを、先に体験する

自分のアプリの半年後を早送りで体験し、
起きた問題を自分で判断して直すアプリです。

誰のため

AIで作れるようになった学生・若手と、迎える企業

何ができる

原因を探して直し方を決めると、その後が変わる

どこが新しい

AIが答えを言わない
題材が自分のコード
判断に結果がつく

使い方

7つのステップ

1 アプリを登録する

2 半年間を早送りする

3 問題が起きて止まる

4 会話しながら原因を探す

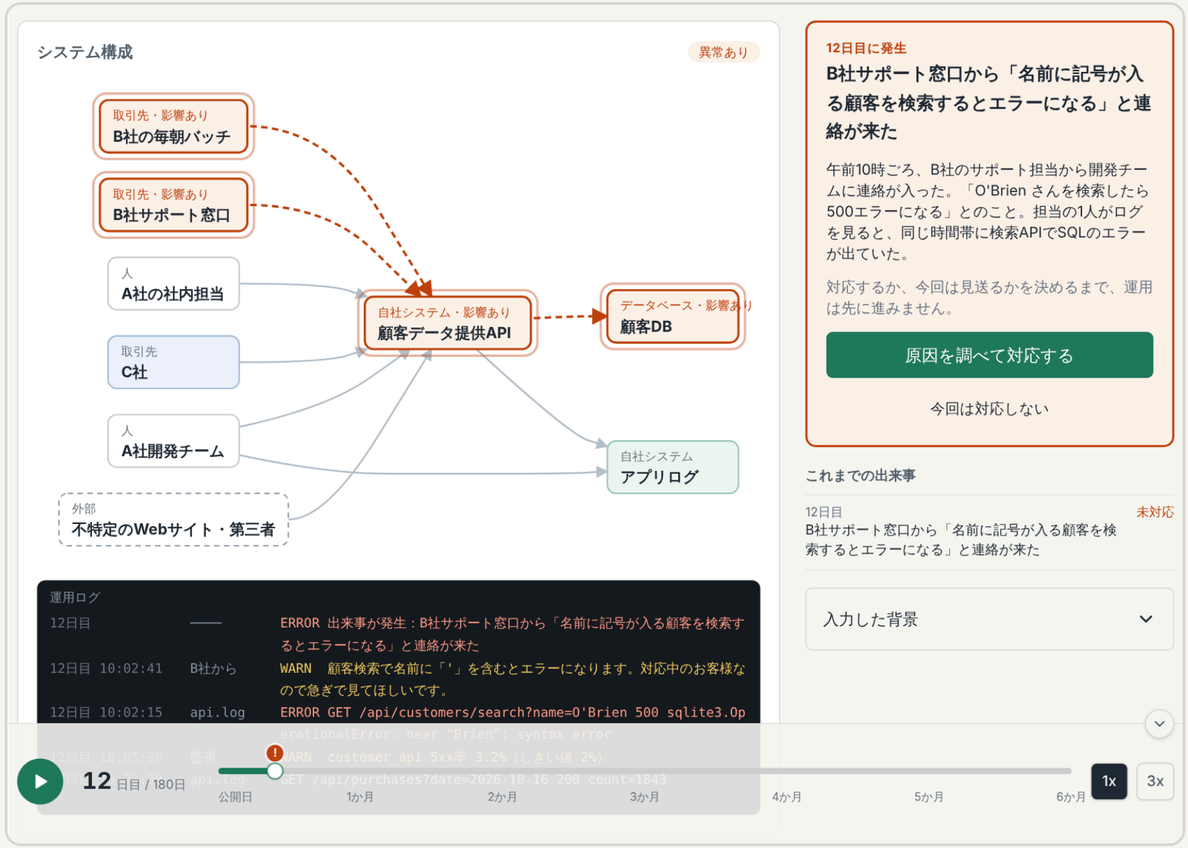
5 直し方を自分で決める

6 その後の運用が変わる

7 判断を記録に残す

実際の画面

半年を早送りして、問題が起きると止まる



シークバーで、公開後の半年間を再生する

構成図で、どこに影響が出ているかが見える

起きる問題の原因は、すべて**自分のコードの中**にある

特長1

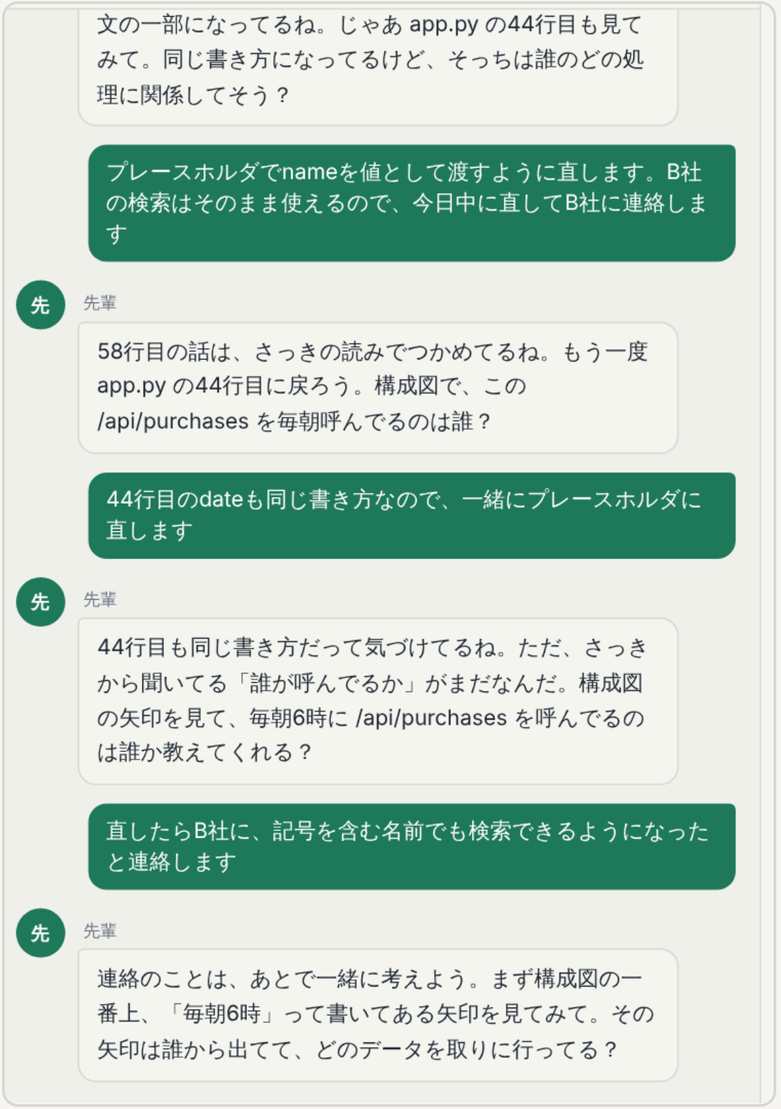
AIは、
答えを言わない

先輩役のAIは、ログやコード、構成図のどこを見ればいいかを問いかけてきます。

こうは言わない

「SQLインジェクションだね。これが正解だよ」

選択肢も出しません。詰まったら、ヒントだけもらえます。



特長2

判断に、結果がついてくる

コードの変化

app.py −2 +3

- dateを文字列連結からプレースホルダに変更
- nameをプレースホルダにし、%は値の側につける

41行目あたり

```
41  41      date = request.args.get("date")
42  42      db = get_db()
43  43      rows = db.execute(
44      -       f"SELECT customer_id, item, price, purchased_at FROM pu
    44  +       "SELECT customer_id, item, price, purchased_at FROM pur
    45  +       (date,),
45  46      ).fetchall()
46  47      logger.info("purchases requested date=%s count=%d", date,
47  48      return jsonify([dict(r) for r in rows])
```

55行目あたり

```
55  56      name = request.args.get("name", "")
56  57      db = get_db()
57  58      rows = db.execute(
58      -       "SELECT * FROM customers WHERE name LIKE '%" + name +
    59  +       "SELECT * FROM customers WHERE name LIKE ?", ("%" + nam
59  60      ).fetchall()
60  61      for r in rows:
```

決めた直し方が**コードに反映**され、修正前後の違いが出る

この先の運用への影響

0 件の出来事が起きなくなる	0 件の影響が小さくなる
0 件の影響が大きくなる	1 件の出来事が新しく起きる

直し方次第で、**この先の半年間**に起きることが変わる

特長3

失敗も、経験として残る



本番じゃないから、間違えていい。判断と理由、次へのメモを残して、次のアプリで見返せます。

ほかの方法との違い

何が違うのか

方法	答えを出すのは	判断したあと
ChatGPTなどの汎用AI	AI	何も起きない
コードの検査ツール	ツール	何も起きない
研修・教材	教材の正解	何も起きない
ジャッジニア	**自分**	**その後の運用が変わる**

しかも題材は、自分のアプリの公開後に起きる問題です。

しくみ

ありえない問題は、起こさない

1

ルールで拾う

コードの中の、該当する行を拾う

2

AIの候補を確かめる

引用した行が実在するものだけ使う

3

直ったか確かめる

修正のあと、もう一度検査する

直したつもりで直っていないときも、そのまま結果に出ます。

解決できること

エンジニアにも、企業にも

エンジニア

- 本番の前に、何度でも判断を経験できる
- AIのスピードと自分の根拠を両方持てる
- 運用を任される中堅へ育っていける

企業

- 新人が障害の前に判断を経験できる
- ベテランの育成の負担が減る
- 判断が一部の人に集中しなくなる

私たちの考え

価値があるのは、
直したコードより判断した経験

セキュリティ対策はイタチごっこで、終わりがありません。
でも、根拠を持って判断する力は、手口が変わっても使えます。

今後の展望

これから広げていくこと

企業向けのサービスに

- 新しい人が入るたびに使える
- 一人ひとりの成長を確かめられる

題材を増やす

- 個人情報・医療など業種ごとの題材
- 自社のコードがなくても練習できる



判断できるエンジニアへ

```
curl -fsSL https://raw.githubusercontent.com/jphacks/tka_2607/main/install.sh | bash
judgineer load
```

この2行で、すぐに試せます。